

Iceberg Ahead: Quick Start Guide

Links

- ML Introduction - <http://bit.ly/4rhc0x2>
- Colab Notebook - <https://bit.ly/4qOAOga>

The Scenario

April 1912, Belfast & London

The unthinkable has happened. RMS Titanic, the pride of the White Star Line, lies at the bottom of the Atlantic. But the crisis isn't over - her sister ship RMS Olympic is still carrying passengers with the same design, and RMS Britannic sits half-built in the Belfast shipyard.

Harland & Wolff and Lloyd's of London have jointly commissioned your team to analyse the survivor data. They need answers: What passenger characteristics most influenced survival? Was it class? Age? Travelling alone versus with family? The findings will drive urgent safety modifications to Britannic and recalculate insurance premiums for Olympic's continued service.

You have 75 minutes!

Get Set Up (5 minutes)

1. Open Google Colab

Go to colab.research.google.com and sign in with your Google account.

2. Open the Starter Notebook

- Download the notebook: <https://bit.ly/4qOAOga>
- Open in colab: File -> Upload Notebook

3. Run the Setup Cells

Run the first two code cells to:

- Download the Titanic dataset
- Install required packages

Click the ► play button or press **Shift + Enter** on each cell.

4. Experiment!

The notebook is yours to explore. Try things out, tweak the code, add new features, and see what happens.

Important: After making changes to your features, remember to re-run the subsequent sections (Train/Test Split → Model Building → Evaluation) to retrain and test your model with the new inputs.

Where to Focus

Section	Focus?	What to Do
1. Data Exploration	✓ Yes	Explore patterns, spot missing values, form hypotheses
2. Feature Engineering	✓ Yes	Create new features, handle missing data, select inputs
3. Train/Test Split	✗ Skip	Already done for you
4. Model Building	✓ Yes	Train your XGBoost model
5. Model Evaluation	✓ Yes	Check accuracy and metrics
6. Advanced Evaluation	○ Optional	Feature importance, confusion matrix

Understanding the Notebook Stages

1. Data Exploration

Load the dataset and investigate what's there — view rows, check data types, spot missing values, and look for patterns.

→ You can't build a good model without understanding your data first. This stage helps you form hypotheses (e.g., "Did women survive more often?") and identify problems to fix (e.g., "Age is missing for 20% of passengers").

2. Feature Engineering

Transform raw data into features the model can use — create new columns, handle missing values, convert text to numbers, and decide which inputs to include.

→ This is where the magic happens. Raw data like "Braund, Mr. Owen Harris" isn't useful to a model, but extracting the title "Mr" tells it something about gender and social status. Good features = better predictions.

3. Train/Test Split

Divide the data into two sets — one for training the model, one for testing it.

→ You need to evaluate your model on data it hasn't seen before. If you test on the same data you trained on, you won't know if the model has actually learned patterns or just memorised the answers. *This section is pre-built — don't change it.*

4. Model Building

Train an XGBoost classifier using your engineered features.

→ This is where the model learns the relationship between your features and survival. XGBoost is a powerful algorithm that works well out of the box, but the quality of your features matters more than tweaking the model.

5. Model Evaluation

Run predictions on the test set and check accuracy metrics — overall accuracy, false positives, false negatives.

→ This tells you how well your model performs on unseen data. It's your scorecard for the competition and helps you understand where the model succeeds or fails.

6. Advanced Evaluation (Optional)

What: Dig deeper with feature importance charts and confusion matrices.

→ Feature importance shows which inputs the model relied on most. The confusion matrix breaks down exactly where predictions went right or wrong. Useful for understanding *why* your model behaves the way it does.

Further Evaluation Ideas

If you want to go deeper, here are some additional ways to evaluate and understand your model:

Tool	What It Shows	Why It's Useful
Confusion Matrix	Grid of correct vs. incorrect predictions by class	See exactly where your model gets it right and wrong
Feature Importance	Ranked list of which features the model relies on	Understand what's driving predictions — are your engineered features helping?
SHAP Values	How each feature contributed to individual predictions	Explains <i>why</i> the model predicted survival or death for specific passengers
Classification Report	Precision, recall, and F1-score per class	More nuanced than accuracy alone — useful when classes are imbalanced
Correlation Heatmap	Relationships between features	Spot redundant features or unexpected patterns in your data
Survival Rate Charts	Bar charts of survival by feature (e.g., by class, gender)	Quick visual check — does the data match your hypotheses?

 **Tip:** Ask Claude or ChatGPT to generate the code for any of these — e.g., "Show me how to plot a confusion matrix with seaborn" or "Generate SHAP summary plot for my XGBoost model".

Feature Ideas to Try

 **Tip:** Not confident with Python? Use an AI assistant like **Claude** or **ChatGPT** to help write your feature code. Describe what you want to create (e.g., "create a feature that groups ages into child, adult, and senior") and let the AI generate the code for you. This challenge is about ideas and experimentation, not coding expertise.

Feature	How	Why
Title	Extract from Name (Mr, Mrs, Miss, Master...)	Social status indicator
FamilySize	SibSp + Parch	Travelling alone vs. with family
FareBucket	Group fares into ranges	Reduces noise, captures class effect
Age (filled)	Replace missing with median	~20% of ages are missing
Cabin Deck	Extract first letter from Cabin	Deck location on ship
IsAlone	1 if FamilySize = 0, else 0	Binary flag for solo travellers

Code Snippets

Pandas Quick Start

```

Python
# View first 10 rows
featurization_df.head(10)

# View column names and data types
featurization_df.dtypes

# Check for missing values
featurization_df.isnull().sum()

# Get summary statistics
featurization_df.describe()

# Count unique values in a column
featurization_df["Pclass"].value_counts()

# Filter rows
survivors = featurization_df[featurization_df["Survived"] == 1]

# Group by and calculate survival rate
featurization_df.groupby("Sex")["Survived"].mean()

```

Feature Engineering Examples

Python

```
# Create FamilySize feature by combining siblings/spouses and parents/children
featurization_df["FamilySize"] = featurization_df["SibSp"] +
featurization_df["Parch"]

# Create FareBucket feature by grouping fares into quantile-based buckets
fare_buckets = 6 # Configurable number of buckets
featurization_df["FareBucket"] = pd.qcut(featurization_df["Fare"],
q=fare_buckets, labels=False, duplicates='drop')
```

Need Help?

- **Stuck on code?** Flag down a roaming Enabler
- **Colab issues?** Try Runtime → Restart runtime
- **Dataset not loading?** Re-run the first setup cell